Here's how I approached the refactor, working through each SOLID principle in turn so the reasoning is traceable back to the specific problems in the original design.

The starting point

```
class Order:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)

    def calculate_total(self):
        return sum(item.price for item in self.items)

    def pay(self, payment_type):
        if payment_type == "credit":
            print("Processing credit card payment...")
        elif payment_type == "paypal":
            print("Processing PayPal payment...")
```

The core issue here is that `Order` has three jobs bundled into one class: managing the cart, calculating totals, and deciding how to process a payment. Martin (2021) makes the point that a class doing several unrelated things becomes fragile precisely because a change to any one responsibility risks breaking the others, and that is exactly what happens when a new payment type means editing the same `pay()` method that also happens to define what the cart even is.

Step 1, Single Responsibility Principle

I pulled payment logic out of `Order` entirely so it only manages items and totals.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price


class Order:
    def __init__(self):
        self.items = []

    def add_item(self, product):
        self.items.append(product)

    def remove_item(self, product):
        self.items.remove(product)

    def calculate_total(self):
        return sum(item.price for item in self.items)
```

`Order` now has one reason to change, how the cart itself works, not how payments are processed or how discounts are calculated.

Step 2, Open/Closed Principle

Since the brief also asks for discounts, I treated that the same way as payments rather than bolting on an if/elif chain. `DiscountStrategy` is an abstract base, so a new discount type is a new class rather than a change to existing code.

```
from abc import ABC, abstractmethod


class DiscountStrategy(ABC):
    @abstractmethod
    def apply(self, total):
        pass


class PercentageDiscount(DiscountStrategy):
    def __init__(self, percentage):
        self.percentage = percentage

    def apply(self, total):
        return total - (total * self.percentage / 100)


class FixedAmountDiscount(DiscountStrategy):
    def __init__(self, amount):
        self.amount = amount

    def apply(self, total):
        return max(0, total - self.amount)
```

This is the same Strategy pattern Ayeva and Kasampalis (2024) describe as one of the more natural fits for OCP in Python, since the abstract base defines the contract once and every variation just implements it differently.

Step 3, Liskov Substitution Principle

For payments, I defined `PaymentMethod` as an abstract base with a single `pay()` method, and every concrete class honours that contract properly rather than throwing errors or behaving unexpectedly.

```
class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount):
        pass


class CreditCardPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Processing credit card payment of £{amount:.2f}...")


class PayPalPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Processing PayPal payment of £{amount:.2f}...")
```

```
class CryptoPayment(PaymentMethod):
    def pay(self, amount):
        print(f"Processing crypto payment of £{amount:.2f}...")
```

Because each subclass genuinely fulfils what `PaymentMethod` promises, any of them can be swapped in wherever a `PaymentMethod` is expected without the calling code needing to know or care which one it got.

Step 4, Interface Segregation Principle

`PaymentMethod` only asks for `pay()`, nothing else. I deliberately avoided the temptation to add things like `refund()` or `get_receipt()` to the same interface, since not every payment method would need them in this context, and forcing every subclass to implement methods it does not use is exactly what ISP warns against.

Step 5, Dependency Inversion Principle

Finally, `PaymentProcessor` depends on the `PaymentMethod` abstraction rather than on any concrete class, and the actual payment method is injected in rather than selected by a string inside the class.

```
class PaymentProcessor:
    def __init__(self, payment_method: PaymentMethod):
        self.payment_method = payment_method

    def process(self, amount):
        self.payment_method.pay(amount)
```

Marcotte and Zebdi (2022) frame this as the difference between a high level module reacting to low level detail versus both sides depending on a shared abstraction, which is what lets `CryptoPayment` get added later without a single line of `PaymentProcessor` changing.

Putting it together

```
if __name__ == "__main__":
    order = Order()
    order.add_item(Product("Keyboard", 45.00))
    order.add_item(Product("Mouse", 20.00))
    order.add_item(Product("Monitor", 150.00))

    subtotal = order.calculate_total()
    discount = PercentageDiscount(10)
    total_after_discount = discount.apply(subtotal)

    for method in [CreditCardPayment(), PayPalPayment(), CryptoPayment()]:
        processor = PaymentProcessor(method)
        processor.process(total_after_discount)
```

I ran this end to end. Subtotal came out at £215.00, the 10% discount brought it to £193.50, and all three payment methods, including `CryptoPayment`, processed correctly through the exact same `PaymentProcessor` class with no modification needed. That last part is really the practical proof that OCP and DIP are doing their job here, since the whole point was that adding a payment type should not mean editing existing, already tested code. Yanakiev, Lazar and Capiluppi (2025) note in their experience report that this kind of extensibility is usually the clearest sign a refactor has actually paid off, rather than just having reorganised the same tightly coupled logic into more files.

References

Ayeva, K. and Kasampalis, S. (2024) Mastering Python Design Patterns: Craft Essential Python Patterns by Following Core Design Principles. 3rd edn. Birmingham: Packt Publishing.

Marcotte, C.H. and Zebdi, A. (2022) An Atypical ASP.NET Core 6 Design Patterns Guide: A SOLID Adventure into Architectural Principles and Design Patterns Using .NET 6 and C# 10. 2nd edn. Birmingham: Packt Publishing.

Martin, R.C. (2021) Functional Design: Principles, Patterns, and Practices. Boston: Addison-Wesley Professional.

Sridhar, S., Indumathi, V. and Hariharan, M. (2023) Python Programming by Pearson. Chennai: Pearson India.

Yanakiev, I., Lazar, B.M. and Capiluppi, A. (2025) 'Applying SOLID principles for the refactoring of legacy code: An experience report', Journal of Systems and Software, 220, 112254.